• Spec-Kit · GitHub Copilot · Azure Functions

A BETTER WAY TO BUILD SOFTWARE WITH AI

Spec-Driven Development

Define **what** and **why** before deciding **how**. Turn requirements into a living spec, a plan, and tasks — then let AI implement against those artifacts, not a throwaway prompt.

Client demo · 2026

THE PROBLEM

"Vibe coding" with AI doesn't scale

- **Intent evaporates.** A prompt is typed, code appears, the "why" is gone forever.
- **Nothing to review.** Reviewers see code with no spec to check it against.
- **Inconsistent output.** The same ask yields different results every run.
- **No traceability.** You can't link a change back to a requirement or ticket.

Prompt → Code

The gap between a one-line prompt and production code is filled with the model's guesses — not your decisions.

```
# today
"add an endpoint to submit orders"
# → 200 lines, unknown assumptions
```

REALITY CHECK

AI is not a compiler

> “I just see things, say things, run things, and copy-paste things, and it mostly works.”
>
> — Andrej Karpathy, on “vibe coding”

- ✗ **Non-deterministic.** Precise input does not guarantee perfect output.
- ✗ **An assistant, not a compiler.** It guesses, and it makes mistakes.
- ✓ **You own the output.** Review, understand, and test everything it generates.

AN ANALOGY

Building a backyard

🔥 “Just build me something cool”

The contractor assumes your intent. The BBQ arrives — but there's no cutout in the stone and no gas line. Every change means tearing up what's already built.

Prompt-and-pray

📐 “The blueprint is the source of truth”

Exact dimensions. Appliances chosen first to define the interface. Utilities laid before pavers. The same blueprint works with any mason.

Human owns “what”, agent owns “how”

Run slow to run fast — invest in the spec to accelerate delivery.

THE IDEA

Make the specification the source of truth

SDD inserts durable, reviewable artifacts between intent and implementation. The AI agent gets structured context at every step — so results are consistent, auditable, and aligned to the requirement.

Write it down

Every feature begins as a `spec.md` — what & why, every requirement testable.

Plan the how

A plan turns requirements into architecture, contracts, and a data model.

Execute

Dependency-ordered tasks drive the AI to build a tested vertical slice.

THE METHOD

One loop, every feature



↺ Repeat **implement → converge** until the gap report says Converged

EACH PHASE FEEDS THE NEXT

Artifacts, not vibes

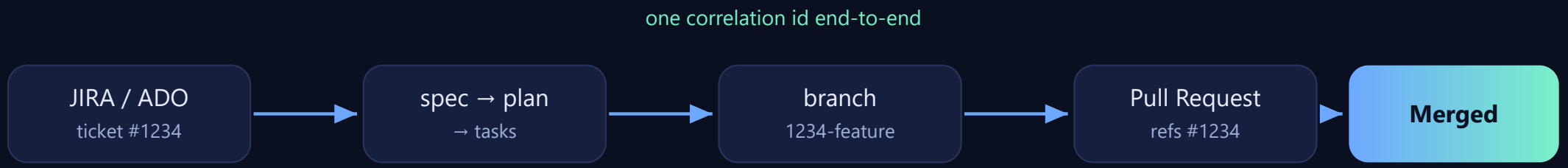
```
specs/001-order-submission/
├── spec.md # what & why
├── plan.md # how — architecture
├── contracts/
│   └── orders-api.md # request/response
└── tasks.md # ordered checklist
```

- **Reviewable.** Each artifact is a PR-sized, human-readable file.
- **Versioned.** The spec evolves in git alongside the code.
- **Structured context.** The AI reads the prior artifact instead of guessing.

FITS YOUR BACKLOG

From ticket to merged PR — one id

Requirements originate in JIRA or Azure DevOps. The ticket id becomes the feature id and the single correlation key across the whole journey.



Inbound

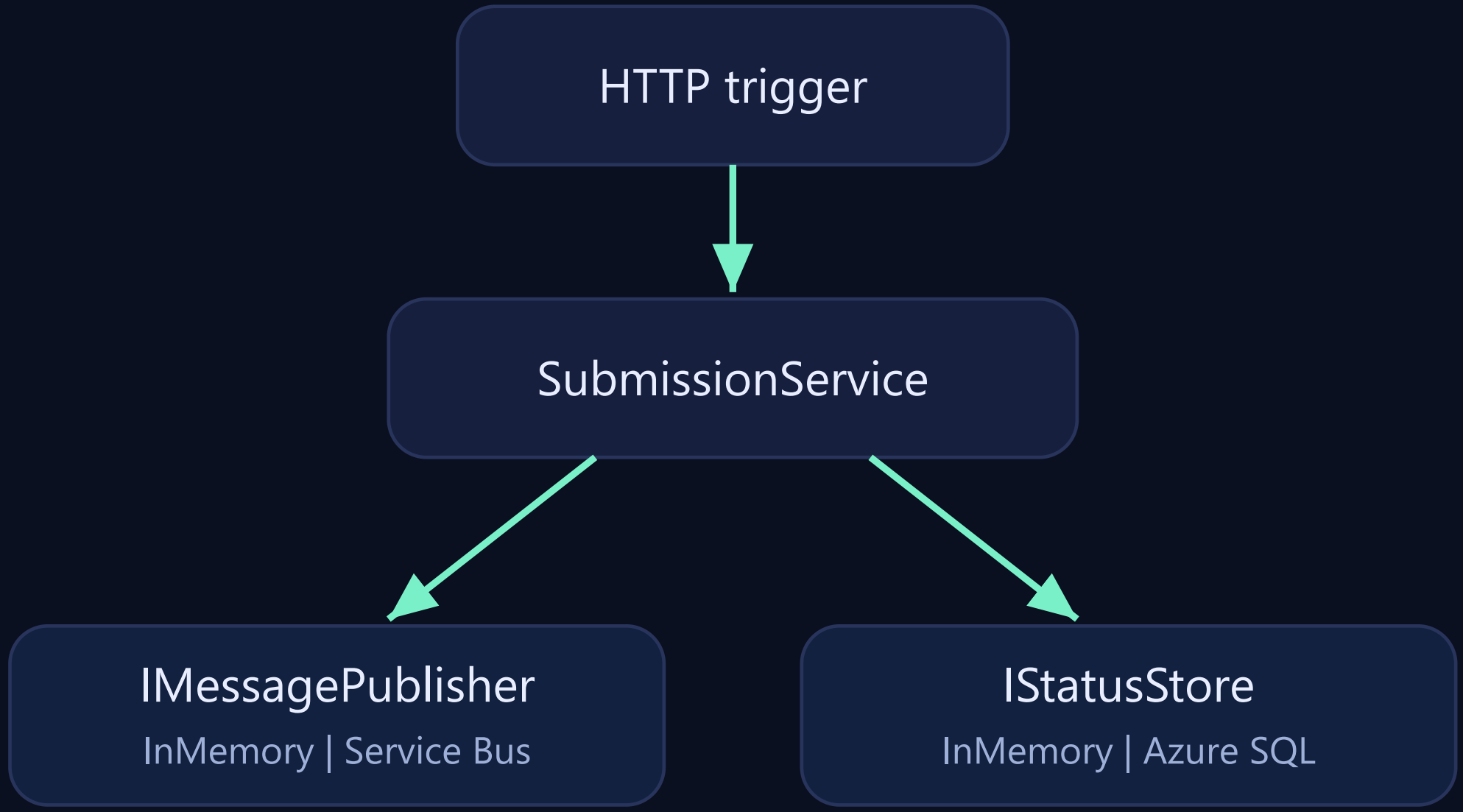
A ticket seeds `spec.md` automatically — never re-typed, always enriched.

Outbound

`tasks.md` creates child work items linked back under the same ticket.

THE DEMO UNDER THE HOOD

Thin adapters, rich core



SEE IT RUN

Live: async order submission

```
# 1 — submit (202 + correlation id)
POST /api/orders
{ "legalEntityId":"ABN-01", "lines":[…] }
→ 202 { "correlationId":"c5d4…" }

# 2 — query status
GET /api/orders/c5d4…/status
→ 200 { "state":"Accepted" }

# 3 — resend same clientRequestId
→ same id · published once (idempotent)
```

- **202 + id** — caller decoupled from slow writes.
- **Idempotent** — safe retries, publish exactly once.
- **Validated** — bad input → 400, nothing published.
- **14 tests green** — all offline.

WHY IT MATTERS

Outcomes, measured

1
source of truth (the spec)

100%
changes traceable to a ticket

0
Azure needed to run the tests

2-line
swap to real Azure services

Shared source of truth

One spec aligns teams and agents on the same intent.

Human at high leverage

Review specs (strategic), not lines of code (tactical).

Small iterations

Frequent spec reviews keep work aligned and prevent drift.

Knowledge & audit trail

Documents the *why*, not just the *what*.

Reproducible outcomes

Same spec → similar code, every run.

AI as collaborator

Let the agent interview you to reach a better spec.

HOW WE RUN IT

The toolchain

Spec Kit

Open-source SDD toolkit — the `/speckit.*` commands & templates.

GitHub Copilot

Runs the loop inside VS Code as slash-command skills.

Azure Functions

.NET 8 isolated worker — the deployable unit per feature.

JIRA + ADO

Backlog in, work items out — full round-trip traceability.

```
## install & go
uvx --from git+https://github.com/github/spec-kit specify init my-app --integration copilot
/speckit.specify · /speckit.plan · /speckit.tasks · /speckit.implement
```

REMEMBER THIS

The 3 separations of SDD

Thinking ≠ doing

Specs define **what**; agents define **how**.

Strategy ≠ tactics

Review **specs**, not lines of code.

Patterns ≠ projects

Build portable **skills**, not one-off prompts.

The 3 practices that matter

Run slow to run fast

Invest upfront to accelerate delivery.

Phases with gates

Validate early, prevent drift.

Let AI interview you

Better specs through collaboration.

AI amplifies what you already have — make sure you're

SOURCES & FURTHER READING

Credits

Spec-Driven Development Using Coding Agents

Arun Gupta, JetBrains — Jfokus 2026. The backyard/blueprint analogy, the 3 separations, the spec-driven SDLC.

Spec-driven Development

Simon Martinelli, JUG Switzerland. “AI is not a compiler”, executable specs, guardrails.

GitHub Spec Kit

github.com/github/spec-kit — the toolkit powering this demo.

Community

ainativedev.io · agentskills.io — patterns, skills, and practices.

CONSTITUTION ONCE · SPECIFY → PLAN → TASKS → IMPLEMENT → CONVERGE PER FEATURE

Spec first.
Then let AI build.

Durable intent, reviewable artifacts, consistent output, and end-to-end traceability — the gold standard for AI-assisted delivery.

▶ Live demo: `func start` + `demo.http`

docs/ · ADRs · constitution